

Especificación técnica y guía de avance

Cada sol donado, verificado con comprobante y evidencia.

AVANCE DEL EXAMEN PARCIAL · CURSO DE INTELIGENCIA ARTIFICIAL

CURSO

Inteligencia Artificial

EVALUACIÓN

Avance de Examen Parcial

ARQUITECTURA

Opción 1 — API SaaS
(Backend + PostgreSQL + LLM API)

SEMESTRE

Académico 2026-II

INTEGRANTES

Nieves Quiñonez, Nicol Tamara
Bonilla Malpartida, Yvan Hawel
Caldas Bahamonde, Arturo Jesús

DOCENTE

Dr. Abimael Adam Francisco Paredes

Estado de la implementación. El aplicativo web está construido y funcionando: 281 pruebas automatizadas en el backend, 20 en el frontend, y el ciclo completo de donación → retención → gasto → verificación → aplicación contable → narrativa al donante.

La capa de IA está diseñada, contratada y con su punto de integración construido y probado, pero la versión actual la resuelve con un motor determinista. La sección 3 distingue qué está implementado y qué requiere conectar la API externa; la sección 7 detalla lo que falta.

01 Definición del problema complejo

El micro-mecenazgo digital en el Perú opera sobre una asimetría de información que ningún actor puede resolver por sí solo: **quien dona no tiene forma de verificar en qué se gastó su dinero, y quien recibe no tiene forma barata de demostrarlo.** La consecuencia no es solo el fraude, sino algo más extendido y más difícil de medir: la desconfianza que impide donar a organizaciones honestas.

Opacidad del destino final del aporte.

El donante entrega dinero a una organización, no a un propósito. Una vez transferido, el sol se mezcla con los demás y su destino se vuelve irrastreable. Los informes de impacto que recibe son agregados, narrativos y no verificables: dicen «ayudamos a 300 familias», no «sus S/ 50 pagaron esta factura». La confianza se pide, no se demuestra.

Costo prohibitivo de la rendición de cuentas para las organizaciones pequeñas.

Una ONG que quiere ser transparente debe digitalizar comprobantes, vincularlos a gastos, verificar su validez tributaria, anonimizar a los beneficiarios que aparecen en las fotos y comunicar todo esto de forma comprensible. Hacerlo a mano consume el tiempo del personal que debería estar operando en campo. Las organizaciones más honestas y más pequeñas son las que menos pueden pagar el costo de demostrarlo.

Verificación humana que no escala y llega tarde.

Auditar cada gasto con comprobante y evidencia visual exige un criterio experto que es caro y lento. Sin apoyo automatizado, o se revisa todo —y el sistema se atasca— o se revisa por muestreo —y la garantía se debilita justo donde más importa—. Las señales de irregularidad son heterogéneas: un comprobante mal formado, una foto reutilizada de otro gasto, un monto fuera de lo habitual para su categoría.

POR QUÉ ES COMPLEJO Y NO COMPLICADO

No tiene una solución técnica única y cerrada. Involucra derecho (Ley N.º 29733 de protección de datos y normativa tributaria de comprobantes de pago), contabilidad (partida doble, trazabilidad a nivel de unidad monetaria), psicología del donante (fatiga comunicacional, fricción de uso), sociología de la confianza institucional e informática. Cada disciplina impone restricciones que las demás no pueden ignorar, y la solución debe satisfacerlas simultáneamente.

02 Definición del aplicativo web básico

Aplicación web modular con backend de API REST y frontend compilado, diseñada para operar bajo un modelo de proveedor externo para las capacidades de IA.

CAPA	TECNOLOGÍA	ROL EN EL SISTEMA
Frontend	Flutter Web 3.47 (Dart 3.13), Riverpod 3, <code>fl_chart</code>	Interfaz única para los cinco roles: donante, administrador de ONG, operador de campo, auditor y administrador de plataforma. El mismo código base compila a Android e iOS sin reescritura.
Backend	NestJS 11 sobre Node.js 22 (TypeScript)	Lógica de negocio, autenticación con JWT y segundo factor, núcleo contable transaccional, y comunicación HTTPS con la API de IA externa . Actúa como proxy seguro: la clave nunca llega al navegador.
Base de datos	PostgreSQL 18	Almacenamiento relacional de usuarios, organizaciones, campañas, fondos, donaciones, gastos, comprobantes, evidencias e historial completo de análisis de IA . Integridad garantizada por <i>triggers</i> , no solo por la aplicación.
Archivos	Adaptador <code>AlmacenamientoArchivos</code>	Comprobantes y evidencias fotográficas, servidos mediante URL firmadas con caducidad. Disco local en desarrollo, S3 en despliegue.

DESVIACIÓN DECLARADA RESPECTO DEL FORMATO

El formato de referencia propone PHP como capa de backend; este proyecto usa NestJS/TypeScript. La decisión es anterior a este avance y está documentada en el Entregable 2 y en los ADR del repositorio.

El patrón arquitectónico de la Opción 1 se cumple íntegramente: el backend custodia las credenciales, consolida el contexto desde PostgreSQL, construye el prompt, consume la API externa por HTTPS y persiste la traza. Lo que cambia es el lenguaje de esa capa, no su función ni su responsabilidad.

03 Requerimientos funcionales con IA

Las tres funciones cubren los tres procesos que el formato distingue: **operativo**, **comunicacional** y **servicio al cliente**. El estado de cada una se declara con la misma escala de tres niveles con que el sistema califica un gasto.

RF-IA-01

● INTEGRACIÓN CONSTRUIDA, API SIN CONECTAR

Verificación automática de gastos

DESCRIPCIÓN

Analiza el comprobante de pago y la evidencia fotográfica de cada gasto y emite un **puntaje de confianza de 0 a 100**, un nivel (ALTO / MEDIO / BAJO) y una **explicación legible en español** de cada señal evaluada. El nivel determina el flujo: ALTO se aprueba automáticamente, MEDIO deriva a revisión humana, BAJO se observa y genera alerta.

SUBPROCESO / ACTIVIDADES

Al registrarse un gasto, el backend encola el trabajo en PostgreSQL. Un trabajador consume la cola, recopila el contexto (datos declarados, hashes de los archivos, historial de la organización, media y desviación de la categoría), lo ensambla en un prompt estructurado junto con las imágenes y lo envía a la API. La respuesta llega en **JSON validado contra un esquema** con los campos `score_documental`, `score_visual`, `score_anomalia`, `score_final`, `nivel`, `datos_extraidos`, `explicacion[]`, `alertas[]` y `version_modelo`. El resultado íntegro se persiste en `analisis_aini`.

COMPONENTE IA

Modelo multimodal con lectura de documentos (OCR del comprobante) y evaluación de coherencia visual, en modo JSON estructurado.

Existe la interfaz `MotorVerificacion` con el contrato de datos completo, la cola, el trabajador, la persistencia, la regla de umbrales y la derivación al auditor. Hoy lo resuelve `MotorReglasV0`, determinista: dígito verificador del RUC por módulo 11, aritmética del IGV, huella perceptual dHash contra todo el histórico y desviación estadística del monto. **Conectar la IA consiste en implementar la misma interfaz y cambiar una variable de entorno.**

Narrativa de impacto personalizada

DESCRIPCIÓN

Cuando un gasto se aprueba y se aplica contablemente a las donaciones que lo financiaron, genera para **cada donante afectado** un mensaje con su monto exacto aplicado, el concepto real del gasto, la organización, la fecha y la evidencia anonimizada. El texto se adapta al aporte concreto de esa persona, no al total del gasto.

SUBPROCESO / ACTIVIDADES

El motor de retorno consulta la aplicación FIFO para saber qué donación financió qué parte del gasto, extrae de PostgreSQL los datos verificados y construye un prompt que **solo contiene campos verificados**, con instrucciones explícitas de no inferir ni embellecer. La IA devuelve asunto y cuerpo; el backend los valida contra un filtro de lenguaje antes de enviarlos.

COMPONENTE IA

Modelo de lenguaje generativo con *system prompt* restringido y validación de salida.

El ciclo completo funciona: una notificación por donante con su monto exacto, un `Proxy` que hace fallar cualquier plantilla que referencie un dato no verificado, y un filtro de lenguaje ético —tolerante a acentos y flexión de género— que rechaza términos como «héroe» o «desgarradora». Sustituir la plantilla por una llamada al modelo exige conservar esas dos validaciones: son las que impiden que el sistema afirme algo que no puede demostrar.

Asistente de transparencia para el donante

DESCRIPCIÓN

Chat integrado en la interfaz del donante que responde preguntas sobre **sus propios aportes**: en qué se gastó su dinero, en qué estado está, por qué sigue retenido, qué respaldo documental existe. También resuelve dudas generales sobre el funcionamiento de la plataforma.

SUBPROCESO / ACTIVIDADES

El donante escribe desde el widget de chat. El frontend envía la consulta al backend, que **recupera exclusivamente los datos del donante autenticado** — sus donaciones, aplicaciones, gastos financiados y estado de cada uno—, inyecta un *system prompt* con las políticas de la plataforma y las restricciones de privacidad, y llama a la API externa.

COMPONENTE IA

Modelo conversacional con *system prompt* delimitado y contexto acotado al usuario en sesión (*guardrails* de contexto y de privacidad).

Es la función que requiere más trabajo nuevo: no existe el módulo de chat ni en backend ni en frontend. Los datos que necesita sí existen y están expuestos: línea de tiempo del aporte, narrativas y aplicaciones FIFO.

RESTRICCIÓN DE PRIVACIDAD QUE CONDICIONA LAS TRES FUNCIONES

El sistema trata datos de beneficiarios de programas sociales, categoría sensible bajo la Ley N.° 29733. Ninguna evidencia fotográfica en la que aparezcan personas puede enviarse a un tercero —incluida la API de IA— sin anonimizar. Esa restricción está implementada **en la base de datos mediante un *trigger***, no en la aplicación, de modo que ningún error de programación pueda eludirla.

04 Base de datos relacional

El esquema tiene **28 tablas de dominio en cinco dominios** (29 en `public`, contando la tabla de control de migraciones). Se detallan las que sostienen la operación básica y las que almacenan las trazas generadas por la IA.

DOMINIO OPERATIVO

TABLA	CAMPOS CLAVE	RELACIÓN / DESCRIPCIÓN
<code>usuarios</code>	<code>id</code> (PK), <code>correo</code> , <code>hash_password</code> , <code>totp_secreto</code> , <code>estado</code>	1 a N con <code>ong_miembros</code> y <code>donantes</code> . Cinco roles vía <code>usuario_roles</code> .
<code>ongs</code>	<code>id</code> (PK), <code>ruc</code> , <code>razon_social</code> , <code>estado_verificacion</code> , <code>puntaje_confianza</code>	1 a N con <code>campanas</code> . El puntaje es público y explicable.
<code>campanas</code>	<code>id</code> (PK), <code>ong_id</code> (FK), <code>titulo</code> , <code>slug</code> , <code>busqueda</code> (tsvector)	1 a N con <code>fondos</code> . Búsqueda full-text en español.
<code>fondos</code>	<code>id</code> (PK), <code>campana_id</code> (FK), <code>categoria_gasto</code> , <code>saldo_recaudado</code> , <code>saldo retenido</code> , <code>saldo_ejecutado</code>	Unidad de trazabilidad. El donante aporta a un fondo, no a una organización.
<code>donaciones</code>	<code>id</code> (PK), <code>donante_id</code> (FK), <code>fondo_id</code> (FK), <code>monto</code> , <code>estado</code>	1 a N con <code>aplicaciones_donacion</code> .
<code>gastos</code>	<code>id</code> (PK), <code>fondo_id</code> (FK), <code>monto_declarado</code> , <code>monto_aprobado</code> , <code>estado</code> , <code>capturado_en</code>	1 a 1 con <code>comprobantes</code> , 1 a N con <code>evidencias</code> .
<code>comprobantes</code>	<code>ruc_emisor</code> , <code>tipo</code> , <code>serie</code> , <code>numero</code> , <code>subtotal</code> , <code>igv</code> , <code>total</code> , <code>validez_cpe</code>	UNIQUE (<code>ruc_emisor</code> , <code>tipo</code> , <code>serie</code> , <code>numero</code>): un comprobante no puede presentarse dos veces.
<code>evidencias</code>	<code>hash_sha256</code> , <code>hash_perceptual</code> , <code>nitidez</code> , <code>contiene_personas</code> , <code>anonimizada</code>	UNIQUE (<code>hash_sha256</code>). El hash perceptual detecta fotos recicladas aunque el archivo cambie.

DOMINIO CONTABLE

TABLA	CAMPOS CLAVE	RELACIÓN / DESCRIPCIÓN
<code>movimientos_contables</code>	<code>fondo_id</code> (FK), <code>secuencia</code> , <code>cuenta_debe</code> , <code>cuenta_haber</code> , <code>monto</code> , <code>hash_previo</code> , <code>hash_actual</code>	Libro de solo inserción. Los <i>triggers</i> rechazan <code>UPDATE</code> y <code>DELETE</code> . Cada asiento se encadena: <code>hash_actual = SHA-256(hash_previo datos)</code> . Un tercero puede recalcular la cadena sin confiar en el sistema.

TABLA	CAMPOS CLAVE	RELACIÓN / DESCRIPCIÓN
aplicaciones_donacion	donacion_id (FK), gasto_id (FK), monto	N a M entre donaciones y gastos. Un <i>trigger</i> diferido exige que la suma aplicada iguale el monto aprobado.

DOMINIO DE INTELIGENCIA ARTIFICIAL

Es el que el formato de referencia resuelve con una columna `ia_log`. Aquí se descompone en cuatro tablas porque la trazabilidad del proyecto lo exige.

TABLA	CAMPOS CLAVE	RELACIÓN / DESCRIPCIÓN
modelos_ia	id (PK), nombre, version, activo	Catálogo de motores. La fila activa hoy es <code>reglas-v0</code> ; al conectar la API se inserta una fila nueva y se comparan métricas contra ella.
reglas_confianza	umbral_alto, umbral_medio, peso_documental, peso_visual, peso_anomalia, vigente_desde	Umbrales y pesos configurables por el administrador. Versionados : cambiar un umbral no reescribe análisis anteriores.
analisis_aini	gasto_id, modelo_id, regla_id, score_final, nivel, datos_extraidos (JSONB), explicacion (JSONB)	Equivalente ampliado del <code>ia_log</code> . Guarda la respuesta íntegra del modelo, atada al motor y a la regla vigente en ese momento.
revisiones_auditoria	gasto_id, auditor_id, decision, comentario, es_muestreo	Decisión humana con comentario obligatorio. Cada fila es una etiqueta supervisada : la versión sin IA ya está generando el conjunto con el que se evaluará el modelo.

05 Procedimiento de integración con la API externa

Flujo técnico bajo el Enfoque Ligero, en cinco etapas secuenciales.

PASO	ETAPA	DETALLE
1	Credenciales y entorno	Se genera una API Key en el proveedor y se almacena como variable de entorno del servidor (<code>AINI_API_KEY</code>), validada al arrancar por un esquema Zod que exige longitud mínima y hace fallar el arranque si falta . La clave nunca se compila en el paquete Flutter ni viaja al navegador.
2	Extracción de contexto (SQL)	Al consumirse un trabajo de la cola, el backend reúne: datos declarados del gasto, hashes y metadatos de los archivos, historial de comprobantes de esa organización, media y desviación de la categoría, y saldo retenido disponible.
3	Construcción del prompt	Se arma la estructura con (a) el rol del modelo —auditor documental, no asistente creativo—, (b) reglas operativas estrictas incluida la prohibición de inferir datos ausentes, (c) el contexto extraído y (d) el formato de salida obligatorio. Las evidencias con personas solo se envían anonimizadas.
4	Consumo REST/HTTPS	Solicitud <code>POST</code> asíncrona con el payload JSON y autenticación por <i>Bearer token</i> , con tiempo límite explícito y reintentos con retroceso exponencial gestionados por la cola.
5	Validación y persistencia	El backend valida la estructura antes de confiar en ella , persiste el análisis completo en <code>analisis_aini</code> junto con <code>version_modelo</code> y la regla vigente, aplica la regla de umbrales y expone el resultado al frontend con puntaje, nivel y motivos legibles.

CONTROL DE EXCEPCIONES

Si la API externa falla, agota el tiempo o devuelve una estructura inválida, el sistema **no se detiene**: la cola reintenta con retroceso y, agotados los intentos, el gasto queda en revisión humana. El motor determinista permanece disponible como respaldo conmutable por variable de entorno.

Un gasto nunca se aprueba por un error de la IA, y nunca se pierde por su caída.

Backend — NestJS + PostgreSQL

Custodia de seguridad.

Es el único que conoce la API Key; el navegador jamás la recibe. La autenticación usa token de acceso corto en memoria y token de refresco en cookie `httpOnly`, con segundo factor obligatorio para los roles que mueven dinero o aprueban gastos.

Consolidación de contexto.

Transforma los datos relacionales planos en objetos estructurados aptos para el modelo, incluyendo agregados que la IA no podría calcular por sí sola: media histórica de la categoría, distancia de Hamming contra el histórico de evidencias, saldo retenido disponible.

Control de excepciones.

Captura latencia, caída o respuesta malformada y devuelve un comportamiento por defecto —derivar a revisión humana— que garantiza que la plataforma siga operando.

Integridad que no depende de la aplicación.

El libro contable rechaza modificaciones por *trigger*; la suma aplicada debe igualar el monto aprobado; una notificación no puede referenciar una evidencia sin anonimizar. **Ningún error de la IA ni del backend puede violar estas reglas**, porque no viven en el código de la aplicación.

Frontend — Flutter Web

Eventos de usuario.

Registrar un gasto con su comprobante y evidencia, aprobar u observar desde la bandeja del auditor, consultar el detalle de un aporte.

Consultas asíncronas.

Peticiones HTTP no bloqueantes hacia las rutas del backend mediante `dio`, con el token de acceso en cabecera y errores traducidos a lenguaje comprensible.

Renderizado dinámico.

Puntaje de confianza con su nivel y motivos legibles, línea de tiempo del aporte (Donado → Retenido → En verificación → Ejecutado), tablero de indicadores con gráficos, y narrativa de impacto con su evidencia.

Accesibilidad declarada.

El lienzo de Flutter Web no expone sus elementos al lector de pantalla hasta que se activa el modo accesible. Por eso las cifras de los gráficos se repiten como texto y el código QR del segundo factor viene acompañado de la clave escrita.

07 Estado frente al requisito evaluativo

«Los alumnos deberán presentar la ejecución funcional de estos 6 puntos en el aplicativo web, demostrando que al menos las 3 funciones de IA consumen la API externa y reflejan los datos dinámicamente en la interfaz.»

PUNTO	MATERIA	ESTADO
1	Problema complejo	● COMPLETO Definido y sustentado en el Entregable 2.
2	Aplicativo web básico	● COMPLETO Construido y funcionando: 281 pruebas en backend, 20 en frontend, cinco roles operativos.
3	Tres funciones con IA	● PENDIENTE Diseñadas y contratadas; ninguna consume todavía la API externa.
4	Base de datos relacional	● COMPLETO 28 tablas de dominio desplegadas, con las cuatro de traza de IA creadas y en uso.
5	Procedimiento de integración	● COMPLETO Definido; el punto de integración está construido y probado.
6	Funcionamiento arquitectónico	● COMPLETO Backend y frontend operativos, con control de excepciones.

Lo que falta, dicho sin rodeos

Las tres funciones de IA no consumen todavía una API externa. La versión actual resuelve la verificación con un motor determinista y la narrativa con plantillas, y el asistente conversacional no existe.

Esto no es un retraso imprevisto: **fue una decisión de diseño explícita** para construir primero el ciclo de confianza completo y verificable, y conectar la IA sobre una base que ya funciona.

Lo que esa decisión hizo posible

Existe la interfaz `MotorVerificacion` con el contrato de datos literal que consumirá el modelo, la cola asíncrona con reintentos, la persistencia versionada del análisis, la regla de umbrales, la derivación al auditor y el registro de cada decisión humana. Conectar RF-IA-01 consiste en implementar una interfaz existente y cambiar una variable de entorno, sin tocar la base de datos, el núcleo contable ni el frontend.

Y hay un efecto que conviene subrayar: como cada decisión de auditor se guarda con su comentario y el nivel que el motor había propuesto, **la versión sin IA lleva meses generando el conjunto de datos etiquetado con el que se evaluará el modelo**. Es la diferencia entre conectar una API a ciegas y conectarla teniendo contra qué compararla.

TRABAJO RESTANTE PARA CUMPLIR EL REQUISITO

FUNCIÓN	TRABAJO	DIFICULTAD
RF-IA-01 Verificación	Implementar <code>MotorAIni</code> contra la interfaz existente: cliente HTTP, prompt, validación de esquema.	● BAJA
RF-IA-02 Narrativa	Sustituir la plantilla por llamada al modelo, conservando el filtro ético y la validación de datos verificados.	● BAJA
RF-IA-03 Asistente	Módulo nuevo en backend y widget de chat en frontend. Es la única sin base previa.	● MEDIA

Anexos y evidencia en el repositorio: matriz de trazabilidad RF/RNF con la prueba que respalda cada requisito · revisión OWASP ASVS nivel 2 capítulo por capítulo · guion de demostración de 10 minutos con los cinco roles · guía de despliegue · seis decisiones de arquitectura documentadas (ADR) · documentación OpenAPI generada del código.

Ayni implementa el proyecto académico *Trazabilidad Radical* — Entregable 2: 21 casos de uso, 21 RNF, 47 RF y 26 tablas en 5 dominios.